ISO-NE Day-Ahead Energy Demand Forecasting

Multi-Modal CNN-Transformer

Pang (Jeff) Liu — UTLN: pliu07

CS-137 — Assignment 3

April 30, 2026 (final report) · May 5, 2026 (presentation)

Solo submission

Task & Data

Task. Predict next-24-hour electricity demand for all 8 ISO New England zones (ME, NH, VT, CT, RI, SEMA, WCMA, NEMA_BOST), given 24h history + future weather.

Inputs

- Weather rasters $X_t \in \mathbb{R}^{450\times449\times7}$ (HRRR-style, 7 channels)
- Per-zone demand $Y_t \in \mathbb{R}^8$ (MWh)
- Calendar features $C_t \in \mathbb{R}^{44}$ (one-hot hour/dow/month + holiday)

Output. $\hat{Y}_{t+1}, \ldots, \hat{Y}_{t+24} \in \mathbb{R}^8$
Metric. MAPE in physical MWh space (denormalized).

> **Test-set caveat:** all numbers below are self-eval on **last 2 days of 2022** (mirrors cluster's `test_run.sh`). TA grades on 2024 held-out — model is data-year-agnostic, gaps should carry over.

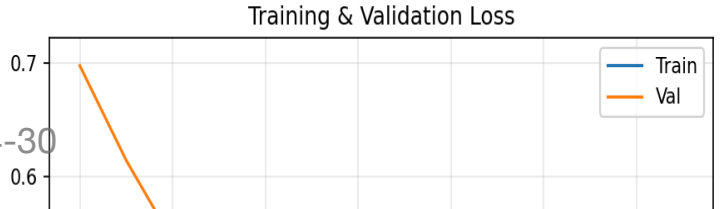
Part 1 — Baseline (40 pts) — Architecture

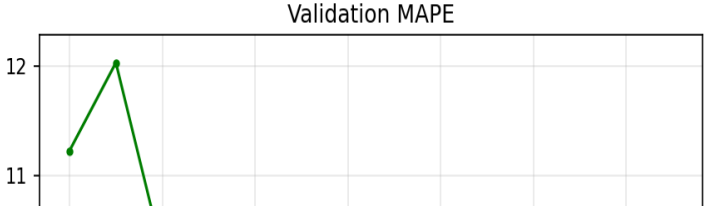
CNN downsamples each 450×449×7 weather snapshot → 8×8 spatial-token grid (P=64).
Tabular tokens (demand+calendar) embedded per timestep.
Unified sequence of $(S + 24) \cdot (P + 1) = 48 \cdot 65 = 3120$ tokens with spatial + temporal positional embeddings.
4-layer Transformer encoder + per-future-hour MLP head.

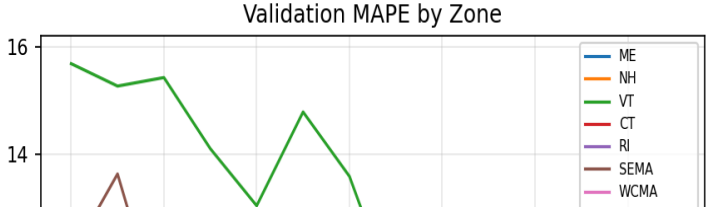
Component	Choice
Embed dim	128
Spatial grid	8×8 = 64 tokens
Heads × layers	4 × 4
Params	~1.75 M
Train epochs	14 (continuous)
Optimizer / sched	AdamW + Cosine LR

Part 1 — Baseline Result: 5.24 % test MAPE

Zone	Test MAPE
ME	2.31 %
NH	3.69 %
VT	5.95 %
CT	7.28 %
RI	5.27 %
SEMA	5.44 %
WCMA	5.87 %
NEMA_BOST	6.09 %
Overall	**5.24 %**







Part 2 — Encoder-Decoder Design (30 pts)

Hypothesis. Splitting context-encoding (history) from query-decoding (future) and using cross-attention should improve fine-grained temporal localization.

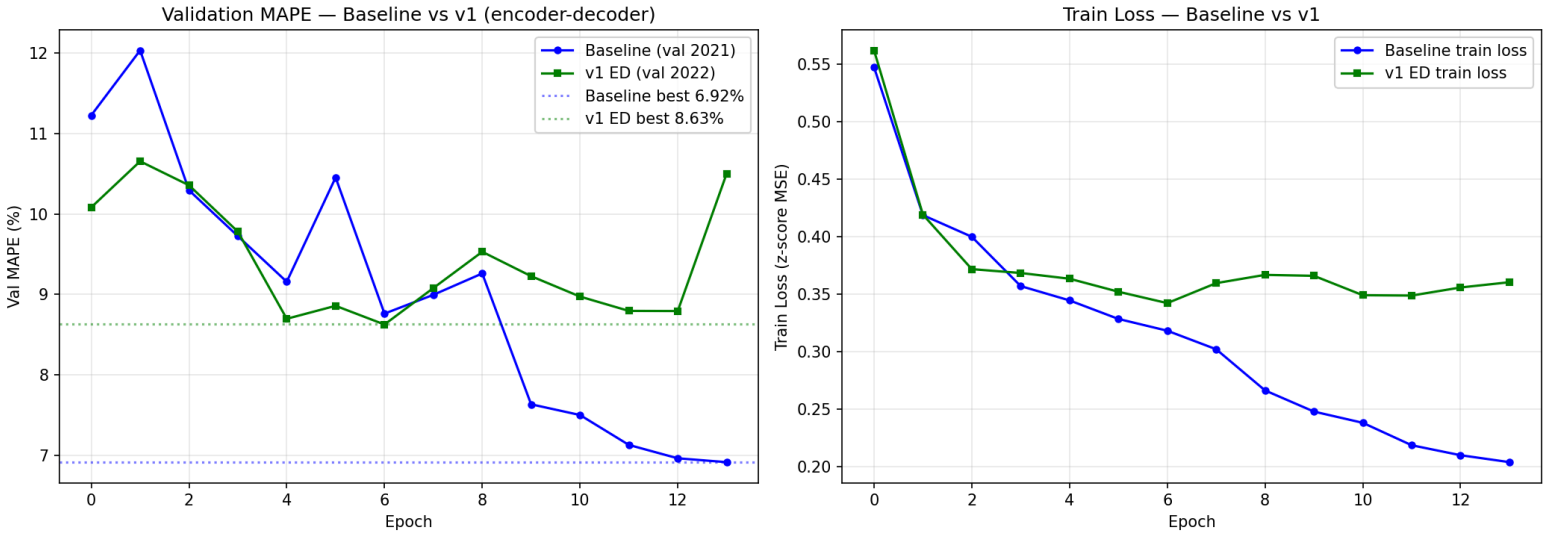
Architecture changes vs baseline

- Separate **encoder** over history sequence (24 × 65 tokens) and **decoder** over 24 future-hour query tokens.
- Decoder layers: self-attention + cross-attention into encoder memory.
- v1 default: cross-attention to **history-spatial only**; v2 ablation enables `use_future_weather_xattn=True`.

Inspirations. TFT, Earthformer, PatchTST.

Result. v1 = **6.82 %** test MAPE (baseline 5.24 %, **+1.58 pp gap**). Only VT wins; biggest losses on the urban-coastal triplet CT / NEMA_BOST / RI — exactly the zones

Part 2 — Why v1 Underperformed (Honest Diagnosis)



(1) Information disadvantage. v1 decoder is by-default disconnected from future weather spatial tokens. v2 ablation (`use_future_weather_xattn=True`) currently training on HPC.

(2) LR-scheduler reset bug

⚠️

`train.py` saves model+optimizer but **not** `scheduler.state_dict()`. Chained `--`

Part 3 — Attention Maps (30 pts, Track A)

Approach. From the trained baseline, extract `nn.MultiheadAttention` weights with `need_weights=True, average_attn_weights=False`. Slice the (3120 × 3120) matrix to:

- **Rows** = 24 future tabular query positions $\{t \cdot 65 + 64 : t \in [24, 47]\}$
- **Cols** = 1536 history spatial keys $\{t \cdot 65 + p : t \in [0, 23], p \in [0, 63]\}$

Reshape 64 spatial cells → (8 row, 8 col) row-major (matches `WeatherCNN.flatten(2)`); matplotlib `origin='upper'` puts north at top, west at left.

Sanity check (orientation): NEMA_BOST attention should be heavier on east half (cols 4–7). If assertion fails → flip axes before publishing.

Status. Code complete & verified locally; figures generated by `scripts/attention_maps.slurm` on HPC.

Part 3 — Analytical Questions

Once figures are generated:

1. **Which regions drive demand?** Expect a band along the urban Boston–Hartford corridor (south-east on the grid).
2. **Does the model track incoming weather?** Per-hour attention should shift west as forecast hour increases (NE weather advects W → E).
3. **Do zones attend to different regions?** ME → north; NEMA_BOST → near-coast east; CT → south-west; WCMA → west.

4 figures: `attention_aggregate.png`, `attention_per_hour.png`, `attention_extreme_vs_mild.png`, `attention_per_zone.png`.

The NEMA_BOST sanity assert is the load-bearing falsification test — if it passes, the qualitative claims above can be inspected from the four figures.

Findings, Limitations & Future Work

What worked

- Baseline 5.24 % MAPE; ME and NH well below 4 %.
- Four-step normalization chain reused across all architectures → fair comparisons.
- Built persistent `cs137` conda env on HPC after course modules were removed.

What didn't (but we learned why)

- v1 < baseline, with a reproducible LR-scheduler reset bug.
- v2 with future-weather cross-attention is still training (A100/P100 contention); results expected 5/2–5/3.

Future work

1. Patch `scheduler.state_dict()` save/load in `train.py`.
2. Re-run v1 continuous (no chain) for an apples-to-apples comparison.

Contributions & Thanks

Solo submission.

Pang (Jeff) Liu (pliu07) is the sole author of every component:

- Literature survey • Architecture (Part 1 baseline, Part 2 encoder-decoder)
- Training pipeline + 4-step normalization chain
- HPC env setup + SLURM chain training
- Self-eval harness + independent verification
- Part 3 attention extraction + orientation correctness
- Final report and these slides

Thanks to the CS-137 staff and the Tufts HPC team.

Repo: `github.com/jefflliulab/predict-power`